UNIT: a type checker for dependent type theory

Thierry Coquand

Abstract

We present UNIT, a type checker for a dependent type theory with Π, Σ, records, data types with primitive recursion, an identity type and cumulative universes, and a module system in the tradition of AUTOMATH: a module is a list of declarations, its parameters are the declarations without a body, and an instance binds the parameters without copying the body. Definitions are treated uniformly. A definition keeps one identity wherever it is used; two uses are compared by their names and arguments before anything is unfolded; otherwise one definition is opened at the head of the comparison, the one declared last. There is no unification and no unfolding heuristic. A consequence is that the work of checking can be observed: UNIT reports, for each declaration, which definitions it opened, and this can be compared with the informal proof. The report exposes computational work whose mathematical justification or representational origin must be explained; it is a diagnostic, not a test of fidelity. We describe the system and this method, and report on its use on a model of univalent type theory and on synthetic algebraic geometry.

1 Introduction

Large proofs are now checked in Lean, Rocq and Agda. The basic ideas of these systems go back to AUTOMATH: propositions as types, proofs as terms checked in a context, and definitions that are not expanded in the proof but expressed by convertibility. That definitions belong to the calculus, and are unfolded by a reduction of their own, is the point of de Bruijn's ΛΔ [12], where a definition is a declaration of the same kind as an abstraction and can occur anywhere in a context. Here we investigate a particular combination: shared representations of definitions, a uniform conversion strategy, and reports of the definitions unfolded, compared with an accompanying mathematical proof.

These systems have also added to pure dependent type theory notions of their own: for instance unification and implicit arguments, type classes or instance search, coercions, tactics, impredicative propositions and definitional proof irrelevance. Each serves a purpose in large-scale formalisation, but they make the work of checking harder to relate to the proof being checked, which is what we want to study here, and they do not seem necessary for representing mathematical arguments. We therefore work with a small dependent type theory with definitions, modules and records, in which nothing is guessed and every unfolding of a definition can be observed. In particular there are no tactics, as in Rocq and Lean: a proof is the term that is checked; and there are no placeholders to be refined interactively, as in Agda: a file is checked as it is written.

This paper presents such a system, UNIT (Section 2). Its type theory has Π, Σ, records, data types with primitive recursion, an identity type, and cumulative universes indexed by level expressions. Its module system follows the segments of AUTOMATH: a theorem is stated in a context, and to use it one gives an instance of the context, the objects and the proofs of the hypotheses, in order; an instance only binds the parameters, and the body is neither copied nor checked again. Its treatment of definitions is uniform: a definition is a declaration like the others, it keeps one identity wherever it is used, and conversion tries to recognise two values by their names before it opens anything, and then opens one definition at a time.

This uniformity is not only a matter of simplicity. In [11] the author recalls translating the first lemmas of Frege's *Begriffsschrift* into AUTOMATH notation, and observes that "these notations represented rather faithfully what is going on when one is trying to understand a proof". We ask whether this holds for the work done during checking too. We consider detailed, precise informal proofs that identify the named constructions and lemmas used and explain the necessary openings of definitions. Ordinary texts often leave such openings silent; making them explicit is part of the comparison, not an assumption about all mathematical reading. We make two hypotheses about these selected aspects of checking.

(H1) *Checking can follow reading.* On a proof accompanied by such a text, a checker can recognise repeated uses of named constructions without unfolding them, and its necessary openings of definitions can be accounted for by the text or by a stated representation convention. This concerns recognition and unfolding; substitution, universe constraints, scope checking and argument recovery remain additional work of the checker.

(H2) *Cost is a diagnostic.* When a step that is routine on paper is expensive to check, the report can help locate repeated decoding of a representation, a missing interface or intermediate statement, or unnecessary work in the checker. This is a heuristic, not an exhaustive diagnosis or a complexity bound: some proofs do compute.

For (H2) to be usable the checker must not hide the cost: a checker with clever unfolding heuristics can make a badly structured proof cheap, and so lose the information. A uniform checker makes its work observable. This should be relevant to the problem of proofs *lost in translation* [4]: a formal proof can be accepted without being the proof of the text it claims to translate. Unfolding reports supply evidence for investigating this correspondence. An unexplained opening may be representational work within the same mathematical argument; agreement of reports cannot exclude a different argument or an incorrectly translated statement.

The method (Section 3) uses this. It has three parts. (1) The checker reports, for each declaration, how its comparisons were settled and which definitions it opened. (2) The informal proof is written, and each step is classified by what a reader does: match names, open a definition the text cites, or nothing. The definitions the text cites can be given to the checker, which then reports the others. (3) For a declaration that costs more than its text suggests, the formal proof is split into the steps of the informal proof, each step is measured, and the expensive comparisons are traced to the part of the checker that asks for them. The method can be applied to an existing development, or while writing a new one.

The aim (Section 5) is that the structure of a formal proof be that of a detailed, precise informal proof: one definition for each notion, one declaration for each step, the intermediate claims written, "by definition" where a definition is opened, and the facts that a text states with a definition proved once by `refl`. That a formal proof is accepted says nothing about whether it is the proof of a given text [4]; the method gives a checkable part of this question.

Results (Section 4) are preliminary. The method found four places where the checker did more than a reader, and places where the informal proof leaves a step to the reader. Table 1 gives the effect on the model of univalent type theory.

2 The checker

2.1 Units and segments

A *unit* is a declaration:

$x\,(\Delta) : A$	a parameter
$x\,(\Delta) : A = t$	a definition
`data` $D\,(\Delta) = c_1(\Delta_1) \mid \ldots$	a data type
`level` a, `constraint` ψ	levels and constraints
`module` $M\,(\Delta)\,\{\,\vec{u}\,\}$, `record` $R\,(\Delta)\,\{\,\vec{u}\,\}$	a segment
`module` $N = M[\vec{a}]$	an instance
`opaque` f, `transparent` f	the unfolding strategy

Terms are those of a dependent type theory with Π, Σ, an identity type with its eliminator J, labelled sums with their eliminators (`split`), records, cumulative universes `U` l indexed by level expressions, and products over levels and over constraints between levels [2].

A file, a module body, a record body and a local block are all lists of units, checked in the same way: each unit in the context of the units before it, and then added to it; a definition stays a definition, with its body available for computation. The scope is sequential: a unit sees only the units before it, never itself or a later one. This is the block structure of AUTOMATH, and, as in $\Lambda\Delta$ [12], a context contains definitions as well as declarations, in any order. The context after a unit does not depend on whether the unit checked: a definition enters the context with its type and its body, not with its proof of correctness. So each declaration can be checked separately, with its own time budget, and one file can be checked while the others are trusted, as a reader accepts a lemma and reads on.

A *segment* is a named list of units; its parameters are its units without a body. It is checked once, with its parameters unknown. An instance $M[\vec{a}]$ checks each a_i against the type of the ith parameter with the earlier ones substituted; nothing of the body is checked again. A module and a record are the same object and differ only in what is given: an instance gives every parameter; a record type $R[\vec{p}]$ gives those of the header, and `new` $R[\vec{p}, \vec{v}]$ gives the others, its fields. Fixing a field, $R[\vec{p}][x := e]$, gives the type of the elements whose field x is e (manifest fields [10]). A hypothesis can be a parameter of a whole development and be discharged by one instance: the model of univalent type theory below assumes function extensionality and univalence as two parameters, and one instance, which checks two arguments, gives it under univalence alone.

Data types are strictly positive. There is no recursion by name: a function out of a data type is a `split`, and a branch names, after its pattern variables, the recursive results, which are hypotheses like any other:

```
data N = zero | suc (n : N) ;
plus (n : N) : N -> N = split { zero -> n ; suc m | r -> suc r } ;
```

This is primitive recursion at higher types, as in Gödel's system T, written as the inductive case expressions of Burstall [5], where termination follows from the scope rules.

2.2 Structures without instance search

Lean finds structure arguments by instance synthesis, and Rocq solves some unification problems on record projections by canonical structures; both choose automatically an argument that could be given explicitly. UNIT has neither. A mathematical structure is a record, generic mathematics about it is a module whose parameter is an element of the record, and `open` gives the operations and laws short names:

```
record CRing {
  R : U a ;  z : R ;  o : R ;
  add : R -> R -> R ;  mul : R -> R -> R ;  neg : R -> R ;
  addC (x y : R) : Id R (add x y) (add y x) ;
  addZ (x : R) : Id R (add x z) x ;  ...
} ;
module RL (Rg : CRing) {
  open Rg ;
  unit (x : R) : U a = (y : R) * Id R (mul x y) o ;
  zl (x : R) : Id R (add z x) x = trans (addC z x) (addZ x) ;
  ...
} ;
```

The ring is given once, for a whole development, as a text says "let R be a commutative ring" and then uses its operations; an instance `RL[Rg]` makes all the results available for a particular ring. The choice is explicit, but it is made once, not at each step.

The difficulty that instance search addresses arises when a proof moves among many related structures: a ring, its additive group, a module over it, a ring of polynomials. With explicit structures each passage is a named construction, and the same structure reached by two constructions must be identified explicitly. In the cohomology of synthetic Stone duality, the groups of cocycles are both kernels of homomorphisms and groups of the Čech complex; statements written with one name and used with the other showed in the report as repeated openings (one declaration opened 3708 definitions), and stating the types in the terms of one construction removed them (447). In the developments of Section 4, records, parameterised modules and `open` have sufficed to organise the structures without instance search, and supplying structure arguments at the level of a development has introduced little repetition.

2.3 Definitions as closures

Evaluation maps terms to values in an environment. Following $\Lambda\Delta$, a definition is not replaced by its body when it is declared; it is unfolded only where its body is needed. The use of closures comes from [6, 25], and was used in the implementation of `cubicaltt` [7], which also introduced `opaque` and `transparent`. A use of a definition f evaluates to the closure $\langle f; \rho \rangle$ of the unit and the environment ρ where it is declared. The *arguments* of a use are the values in ρ of all the variables and levels in scope at the declaration. A member $M[\vec{a}].f$ of an instance is $\langle f; \rho, \vec{a} \rangle$. Nothing is copied: f has one identity wherever it is used.

`opaque f` declares that from this point on f is not unfolded, and `transparent f` undoes it. The strategy is a parameter of the run: evaluation builds the same values whatever the strategy. Making a definition opaque can only make fewer terms convertible, so an `opaque` declaration never makes an incorrect proof correct. Its use is the mathematical one: once a construction is characterised by lemmas, the rest of the development uses the lemmas.

2.4 Conversion

Definitional equality is generated by β (for functions, pairs, levels, projections, `split` on a constructor and J on reflexivity), the unfolding of definitions, and η for functions and pairs. Two values are compared as follows.

1. *By names.* They are equal if they are strictly equal, that is, equal as represented, two closures $\langle f; \rho \rangle$, $\langle f; \rho' \rangle$ being strictly equal when ρ and ρ' are; or if they are eliminations of the same definition f with convertible arguments and spines. Nothing is unfolded.

2. *By head steps.* Otherwise the β-redexes at the head are reduced, without unfolding. If a definition blocks the head of one side only, that side is brought to weak head normal form. If both heads are blocked by definitions, the one declared later is opened by one δ-step. After each step, strict equality is tried.

3. *By components.* When no head is blocked, the outer forms are compared and their components are compared recursively, each from stage 1.

Stage 1 is a reader who recognises the same lemma at the same arguments; stage 2 is opening a definition, the δ-step of $\Lambda\Delta$. The strategy is uniform: there is no other choice of what to open, and no heuristic depending on the definitions. Sequential scope ensures that an earlier declaration's body cannot refer to a later declaration. It does not impose this order on instantiated values: if $f(x) := x$ is declared before $g := a$, then $f(g)$ unfolds to g. The later-first rule is a fixed policy, not a completeness argument. Two heads can also reduce to a common expression. If $h := a$ and then $k := a$, comparison proceeds as

$$h \sim k \quad \longmapsto \quad h \sim a \quad \longmapsto \quad a \sim a.$$

Two occurrences of the same `split` with different environments are compared first at their full function types (domain and motive), then by their branches under fresh variables for the constructor fields and recursive results; two record types are equal when they come from the same declaration with equal parameters, and their fields are never compared.

Checking follows the term. An elimination (an application, a projection) is checked by inferring its type and comparing it with the expected type as it is; the expected type is unfolded only when the term is an introduction (a λ, a pair) and its outer form is needed. A reader does not open the statement of a lemma to check that the lemma has been applied.

Notation. Omitted arguments are filled by one-way matching of the inferred types of the explicit arguments against the domains, never by unification; two different solutions for a hole are an error. A solution is written back as a term in the current scope, recognising the names in scope by stage 1 only: a reader who writes an omitted argument recognises names and does not compute. Equational chains `calc`, `open` and `import` are translated before checking.

3 The method

The measurement. With the option `--opened`, UNIT reports for each declaration the number of comparisons (including subproblems) settled at stage 1, by arguments (stage 1 with a recursive comparison of arguments), and by components (stage 3), and the definitions unfolded during its check, with multiplicities. The unfolding of a data type to its constructors is not counted: it is reading a declaration, not opening a construction. The counters are active while a declaration's checking result is evaluated, including argument matching and writing back inferred arguments when these are forced in that interval. They are not restricted to the final comparison of the inferred and expected types. Parsing and name resolution are outside this interval; diagnostic printing after the result is obtained is also outside it. With lazy evaluation, shared work is attributed to the declaration that forces it. The reported times are CPU times for these intervals. Unfolding counts describe this instrumented execution, not all operations of the checker and not a machine-independent cost of the proof.

What the text opens. In HOL every use of a definition is an explicit step, a rewrite with its defining theorem; in AUTOMATH and in UNIT the uses of a definition are silent. Objective type theory [3] does the same as HOL for computation rules: β and the computation of the

identity eliminator hold only propositionally, with explicit witnesses, so each use is a transport written in the term. An informal text is in between: it says "by definition of X" where it uses a definition, and says nothing at steps it regards as obvious. Frege had no definitional equality for defining notions: in the *Begriffsschrift* [16] a definition is a judgement of identity of content, and each use of it is a step citing it. Each step of a text is classified by what a reader does:

step of the text	the reader	the checker should open
by Lemma L	matches names	nothing
by definition of X	opens X	X
without comment	nothing	nothing

With `--opens F`, where each line of `F` has the form `name:` d_1 `...` d_k, UNIT prints for the declaration `name` the definitions it opened other than $d_1, \ldots, d_k$. Each is a use not listed in the supplied annotation. It may implement a representational convention or an ordinary silent use of a definition; its origin must be investigated before drawing a conclusion about the mathematical argument.

Reading a declaration. For a declaration that is slow, or opens many more definitions than its text cites:

1. write its informal proof;
2. say how a reader checks each step: by a lemma, by a definition, or without comment;
3. split the formal proof into these steps, one lemma for each step with its intermediate points written, and measure each;
4. for the step that costs more than its text, find the comparisons that cost, and the part of the checker that asks for them (a traced version prints each comparison costing more than a threshold with its caller: subsumption, a `refl`, the agreement of two solutions of a hole, the writing back of a solution, the arguments of the same definition, a `split`, a locked definition).

The diagnosis is one of three: the checker departs from reading (repair the checker); the formal proof names one notion in several ways (repair: one name); or it uses a fact that the text states, or makes by notation, without stating it (repair: a lemma, stated once at the generic level).

Writing. The same method applies while writing, and gives rules: write the informal proof first; one name for each notion (no alias for something that has a name, pass a structure rather than its components, name the components of a construction once where it is defined); state the facts "by definition" once, by `refl`, next to the definition, and lock the construction after its characterising lemmas; write the intermediate point of a step that the text proves by two definitions; give accessors for the parts of elements rather than chains of projections; after each declaration, compare what it opened with what its text cites.

4 First results

4.1 Small examples

Sharing and the lock. On

```
module Mon (A : U a) (e : A) (op : A -> A -> A) {
  twice (x : A) : A = op x x ; } ;
module NM = Mon[N, zero, plus] ;
```

the report is:

declaration	statement	opened
`twiceUnfold`	`NM.twice x = plus x x`	`Mon.twice`
`byName`	`Mon[N,zero,plus].twice x = NM.twice x`	nothing
`opaque Mon.twice`		
`viaLemma`	`NM.twice one = plus one one`, by `twiceUnfold`	nothing
`transparent Mon.twice`		
`compAgain`	`NM.twice one = suc one`, by `refl`	`Mon.twice`, `plus`, `one`

The same instance reached by two paths is recognised without opening `twice`; after the lock, the lemma `twiceUnfold` is used instead of the definition.

4.2 The developments

The method was applied to four developments.

- A model of univalent type theory over a base category with an interval, with finite posets and simplicial sets as instances, and the localisation at a modality of Sattler (13 475 declarations).
- Synthetic algebraic geometry [24], up to Châtelet's theorem: a Severi–Brauer variety with a point is projective space, from the axioms and a left exact sheafification for the étale topology (15 547 declarations).
- Synthetic Stone duality [22]: Stone spaces and countably presented Boolean algebras up to its Theorem 4.18, and sequential colimits and first cohomology up to its Theorem 5.16 (3264 and 4448 declarations in its two parts, which share some files).
- Cohomology in synthetic Stone duality [23], on top of the previous one: the cohomology of compact Hausdorff spaces, up to the comparison of H^n with Čech cohomology as a natural isomorphism of groups (about 8500 declarations with the files it reuses, checked in under four seconds).

In the first two, about half of the declarations open no definition at all (6325 and 7879). The last two were written from the start by the rules of Section 3, comparing after each file what its declarations open with what their texts cite.

4.3 Where the checker departed from reading

Each of the following was found by a declaration that cost more than its text, and repaired in the checker.

1. *Checking an application against a type.* The first version brought the expected type to weak head normal form before choosing a rule, even for an application whose type is inferred and compared. This showed as a lemma made of two applications of another opening a definition twenty times. With the rule of Section 2.4, it opens nothing, and the model checks in 40.1 s instead of 47.5 s.
2. *Writing back a solution.* In the model, a construction ω builds a square from a path (a *side*) in a presheaf, and a lemma, `sqVN`, states that ω is natural: the square of a side restricted along a map of the base category is the restriction of its square. Its proof has three steps: the side is natural, ω is natural, and the definition of the restriction of a path. The definitional step costs nothing, and the naturality of ω, stated with its

points, costs nothing; the cost was in using it under the evaluation of the square at a point. It came from writing back the omitted arguments as terms: the checker searched the instances in scope for one with the right parameters by full conversion. With the write-back recognising names only, `sqVN` checks in 0.03 s instead of 0.82 s.

3. *The type of an equational chain.* While filling omitted arguments, the checker inferred the type of a chain by checking it again, so that a chain of n steps checked its inner steps up to n times. Reading the type off the motive of the chain removes 16% of the unfoldings of the model; a nine-step chain goes from 0.44 s to 0.16 s.

4. *Matching level applications.* Matching did not recognise two applications of the same level-polymorphic definition as having the same head; with this rule, a step of Proposition 3.3.1 of the Châtelet development goes from 2.06 s to 0.06 s.

Stage 1 of conversion is essential: an experiment that compares arguments only strictly, and otherwise unfolds, takes more than 30 s on each file of the model tried, against 1.5–4 s.

4.4 What the unfolding reports revealed

The following reports exposed representational work and an intermediate identification left to conversion. They suggest changes to the formal presentation; neither observation alone establishes a departure from the informal argument.

An identification the text makes by notation. What remains in most files is the decoding of element types: a text writes an element of the extension $\Gamma.A$ at W as a pair (γ, a) without comment, while the checker opens the code of the extension at each projection. On the naturality lemma `sqVN` above, the definitions opened beyond the one its text cites are 838, of 38 kinds, all of this decoding. The report isolates them as one notion, the representation of the elements of a presheaf by codes. An accessor stated once ("the value of the path at the point (k, j)") reduces such a declaration from 3091 to 697 definitions opened. The accessor provides an interface to the representation: the mathematical argument need not change when its repeated decoding is removed.

A remark proved by a computation. In the cohomology of synthetic Stone duality, the delooping of a homomorphism h is multiplicative on loops because, on loops, it is $\mathrm{band}^{-1} \circ h \circ \mathrm{band}$, and the band is a group isomorphism. The first proof left the first point to conversion inside one comparison, and opened 1576 definitions. Stating it as a lemma, proved by `refl` once, and the second point in the terms of the band, gives two declarations opening 240 and 490. The two mathematical steps are to identify the induced map on loops with the displayed composite, and then to use preservation of multiplication by the three maps in that composite. Stating the identification separately makes this division of the argument visible; the counts measure its effect on checking rather than establishing fidelity to the text.

4.5 Totals

Table 1 gives the totals for the model: for a checker that brings both sides of a failed comparison by names to weak head normal form, as is usual; for the uniform checker, with the corrections 1, 2 and 3 above; and with five files repaired by the rules of Section 3 (one name for each notion), on a copy of the development. These are historical measurements, predating the additional type check for `split` described in Section 7. The exact revisions, machine and build settings for these runs have not been identified in the available measurement record, so the table is preliminary rather than a reproducible benchmark. Current validation, including source hashes, build information and commands, is recorded separately in `doc/review-validation.md`; it

	by components	definitions opened	time
both heads to weak head normal form	3 183 186	7 699 675	40.5 s
uniform, corrections 1–2	1 027 755	1 737 502	27.4 s
and correction 3	966 171	1 454 435	29.8 s
and repairs of five files	632 549	1 229 233	28.5 s

Table 1: Checking the model (13 475 declarations): comparisons settled by components, definitions opened, and time spent in the declarations (CPU time). These historical runs used the experimental branch-only `split` comparison. Times are of single runs and vary by about 10%.

does not reproduce the four historical variants. Counts and runtime measure different things: correction 3 reduces unfoldings while the measured time increases. The Châtelet development checks in 10.4 s with the uniform checker, opening 0.41 million definitions. A recent development of the exterior algebra and determinants (1385 declarations with the files it uses), not written by the rules of Section 3, opens 27 110 definitions with the uniform checker against 45 171; its largest declaration opens 1364, mostly two aliases, `Alg` and `Ext`, of notions that have names.

Several slow declarations of the model have not yet been read: in one, no comparison is costly, and the time is elsewhere; others are decodings of element types, for which a common accessor is proposed but not yet built.

5 Alignment

That a formal proof is accepted does not establish that it is the proof of the text. This is the point of [4], who show that the Lean formalisation accompanying an announced proof of blow-up for the Navier–Stokes equations does not follow the text (some lemmas are proved in a different, weaker form), and argue that a faithful translation of a mathematical text cannot in general be guaranteed by a computation.

The aim here is to make selected aspects of the correspondence inspectable: the named constructions, applications of lemmas and openings of definitions in a formal proof are compared with a detailed, precise informal proof. The following table describes the intended organisation, rather than a criterion deciding fidelity:

informal proof	formal proof
a notion, with its name	one definition, used under that name
a lemma, a step	a declaration
an intermediate claim	a point written in the chain
“by Lemma L”	an application of L, checked by names
“by definition of X”	an opening of X, declared
a fact stated with a definition	a lemma proved by `refl`

The checker exposes computational work whose mathematical justification or representational origin must be explained. An opening absent from the annotation can arise from a second name for a notion, decoding of a representation, an unstated intermediate identification, or an ordinary use of a definition left silent by the text. Each requires examination; none by itself proves that the arguments differ. Conversely, agreement of unfolding reports cannot establish that the statements or arguments correspond. A human reader must still compare both. The report supplies local evidence for that comparison, maintained while the proof is written.

A formal proof is well written, in this sense, when its structure is that of a detailed informal proof, and its openings can be explained by that text and its representation conventions.

The informal proof this asks for is longer than usual: it has the explicit connections between convertible notions, each a lemma proved by `refl`, checked once, and free to use.

6 Related work

Definitions in the calculus. De Bruijn's ΛΔ [12] makes definitions part of the typed λ-calculus, with a reduction of their own, and his *mathematical vernacular* [13] is a language meant to be close to the way mathematics is written. The block structure of Automath is that of Unit. The theories of Half [8] are lists of declarations; Wang [25] implements a type theory with definitions as closures, which Unit follows.

Controlling unfolding. Implementations of type theory decide what to unfold by heuristics: the kernel of Lean [15] unfolds first the definition of greater height, which is close to our rule of opening the later definition, and other systems use priorities set by the user. Opaque definitions, and the controlled unfolding of [19], make unfolding a matter of the theory rather than of the implementation. Here the strategy is kept uniform, and the resulting reports are used to compare unfolding with an accompanying text.

Computation as explicit terms. In objective type theory [3] the only judgemental equality is syntactic identity; the computation rules of Π and of the identity type are propositional equalities with explicit witnesses, and type checking is quadratic. Its rules are interpretable in Unit, each witness becoming `refl` (the unbased eliminator being defined from the based one), but not conversely: Unit has more equalities, hence fewer models, and does not inherit the quadratic bound. The two make opposite choices for the same information. Objective type theory records each use of a computation in the proof term; Unit leaves the term unchanged, lets it serve at a definitionally equal type, and reports which definitions the comparison needed.

Explicit conversion certificates. Carneiro's MM0 [9] also aims at a small, efficient verifier. It separates a readable specification of axioms, definitions and theorem statements from a proof certificate, which can be produced by the MM1 front end. MM0 has a convertibility judgement for unfolding definitions, separate from the assertions of the chosen object logic. Its certificates give explicit conversion steps, including congruence and unfolding, and can share expressions and conversion proofs. Unlike the propositional computation witnesses of objective type theory, these certify the verifier's conversion judgement.

MM0 therefore already makes unfolding explicit and inspectable. The difference is where the unfolding steps are supplied: MM0 checks those in a certificate; Unit chooses them while checking a dependent proof term and reports them. MM0's compiled certificates are intended for efficient verification; our aim is to relate the work on the written proof term to the reading of an accompanying text. Such a comparison could also be made from MM0 certificates, given a correspondence with the source proof. Observability alone is not a distinction between the systems.

Structured formal proofs. Mizar, Isar [27] and Lamport's structured proofs [21] aim at formal texts whose structure is that of an informal proof; this is the aim of Section 5, which adds a measurement of what each step opens. De Bruijn's estimate of the ratio between the length of a formal text and that of the informal one [14] compares sizes, not structures.

Human checking. Ganesalingam and Gowers [17] build a prover whose output reads as a human proof; in it, expanding a definition is a move of its own, of low priority, taken when nothing else applies. Ganesalingam [18] studies the language of mathematical texts, including

definitions. Empirical studies of how mathematicians check proofs [26, 20] find that experts check steps locally, each against its justification, and spend their time on the steps whose justification is implicit. We know of no study of when a human reader opens a definition; the measurement proposed here gives a prediction that such a study could test.

7 Limitations and open questions

The results are preliminary: the earlier measurements concern selected declarations, and the repairs underlying Table 1 were made on a copy. The classification of the steps of a text is done by hand. The table and the individual counts above were obtained with an experimental `split` comparison that compared branches without comparing their types. This was a correctness issue, not an optional extension of the calculus. The current checker additionally compares the full function types of the two occurrences, recovering the domain and motive from the enclosing declaration and the occurrence's path. Nested occurrences retain the actual, possibly abstract, recursive results in their environments. Focused regression checks and reruns of the developments are recorded in `doc/review-validation.md`; the historical measurements have not been re-established for all variants.

We do not give a soundness or completeness proof of the conversion algorithm. Reference [2] supplies the universe discipline; definitions follow $\Lambda\Delta$. The labelled inductive types with primitive recursion, nominal records, fixed fields and module rules described here are additional rules, not all part of [2]. Their combined metatheory remains to be supplied. Other open points include η for records and the restrictions on local units (a local block cannot introduce a data type or a module).

The checker, the examples and the developments are available at `https://github.com/coquand/unit`. The implementation is about 5700 lines of Haskell:

part	lines
syntax, values, evaluation, conversion, checking of units and segments	1850
omitted arguments: matching and writing back	980
universe levels: the decision of constraints [1]	170
parser and resolver	1740
printer, with the read-back of values to terms	170
strict positivity	130
driver, checking server and reports	620

Besides the Haskell base library, the checker uses only the standard packages `containers` (finite maps and sets) and, for the checking server, `directory`; the parser is written by hand, without a parsing library. Each declaration has a budget of ten seconds; a checking server loads a development once and rechecks only the changed files and those depending on them.

Acknowledgements

The author supplied the design of Unit and directed its development and the interpretation of the checking experiments. Claude (Anthropic) produced the implementation code, and Claude and Codex (OpenAI) produced the formalisation code through iterative work under that direction. This attribution concerns code production; the mathematical sources are cited in the text. In particular, the mathematical results concerning the D modality and the D_n modalities, including their localisation and limit properties, are due to Christian Sattler. The present work concerns their formalisation and checking, and the resulting text and software remain the author's responsibility.

References

[1] M. Bezem and T. Coquand. Loop-checking and the uniform word problem for join-semilattices with an inflationary endomorphism. *Theoretical Computer Science* 913, 2022.

[2] M. Bezem, T. Coquand, P. Dybjer and M. Escardó. Type theory with explicit universe polymorphism. arXiv:2212.03284.

[3] B. van den Berg and M. den Besten. Quadratic type checking for objective type theory. arXiv:2102.00905, 2021.

[4] A. Bastounis, F. Circelli and A. C. Hansen. Navier–Stokes lost in translation: why Lean verification of AI autoformalisation does not guarantee correct natural language proofs. arXiv:2610.08144, 2026.

[5] R. M. Burstall. Inductively defined functions in functional programming languages. *Journal of Computer and System Sciences* 34(2/3):409–421, 1987.

[6] T. Coquand. An algorithm for type-checking dependent types. *Science of Computer Programming* 26(1–3):167–177, 1996.

[7] C. Cohen, T. Coquand, S. Huber and A. Mörtberg. Cubical type theory: a constructive interpretation of the univalence axiom. In *TYPES 2015*, LIPIcs 69, 2018. Implementation `cubicaltt`: `https://github.com/mortberg/cubicaltt`.

[8] J. Cederquist. *A Pointfree Approach to Constructive Analysis in Type Theory.* PhD thesis, Chalmers, 1997.

[9] M. Carneiro. Metamath Zero: designing a theorem prover prover. In *Intelligent Computer Mathematics (CICM 2020)*, LNCS 12236, Springer, 2020, 71–88. doi:10.1007/978-3-030-53518-6_5.

[10] T. Coquand, R. Pollack and M. Takeyama. A logical framework with dependently typed records. *Fundamenta Informaticae* 65, 2005.

[11] T. Coquand. Some remarks about dependent type theory. In B. Meyer, editor, *The French School of Programming*, Springer, 2023, 175–202. doi:10.1007/978-3-031-34518-0_8.

[12] N. G. de Bruijn. Generalizing Automath by means of a lambda-typed lambda calculus. In *Mathematical Logic and Theoretical Computer Science*, Lecture Notes in Pure and Applied Mathematics 106, Marcel Dekker, 1987, 71–92. Reprinted in *Selected Papers on Automath*, North-Holland, 1994.

[13] N. G. de Bruijn. The mathematical vernacular, a language for mathematics with typed sets. In *Workshop on Programming Logic*, Marstrand, 1987. Reprinted in *Selected Papers on Automath*, North-Holland, 1994.

[14] N. G. de Bruijn. A survey of the project Automath. In *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, Academic Press, 1980, 579–606.

[15] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *CADE-28*, LNCS 12699, Springer, 2021.

[16] G. Frege. *Begriffsschrift, eine der arithmetischen nachgebildete Formelsprache des reinen Denkens.* Nebert, Halle, 1879. English translation in J. van Heijenoort, editor, *From Frege to Gödel*, Harvard University Press, 1967.

[17] M. Ganesalingam and W. T. Gowers. A fully automatic theorem prover with human-style output. *Journal of Automated Reasoning* 58, 2017, 253–291.

[18] M. Ganesalingam. *The Language of Mathematics*. LNCS 7805, Springer, 2013.

[19] D. Gratzer, J. Sterling, C. Angiuli, T. Coquand and L. Birkedal. Controlling unfolding in type theory. arXiv:2210.05420, 2022.

[20] M. Inglis and L. Alcock. Expert and novice approaches to reading mathematical proofs. *Journal for Research in Mathematics Education* 43, 2012, 358–390.

[21] L. Lamport. How to write a proof. *American Mathematical Monthly* 102, 1995, 600–608.

[22] F. Cherubini, T. Coquand, F. Geerligs and H. Moeneclaey. A foundation for synthetic Stone duality. arXiv:2412.03203, 2024.

[23] F. Cherubini, T. Coquand, F. Geerligs and H. Moeneclaey. Cohomology in synthetic Stone duality. Manuscript, 2026.

[24] F. Cherubini, T. Coquand and M. Hutzler. A foundation for synthetic algebraic geometry. arXiv:2307.00073.

[25] Q. Wang. *A Haskell Implementation for a Dependent Type Theory with Definitions*. Master's thesis, Chalmers, 2021.

[26] K. Weber. How mathematicians determine if an argument is a valid proof. *Journal for Research in Mathematics Education* 39, 2008, 431–459.

[27] M. Wenzel. Isar: a generic interpretative approach to readable formal proof documents. In *TPHOLs'99*, LNCS 1690, Springer, 1999.